

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Trabajo Integrador – Programación 1

Gestor de Países en Python: Persistencia, Filtrado y Análisis de Datos con CSV

ALUMNOS: NAHUEL LEMUS, JUAN ANGLAT

Resumen

El presente trabajo analiza y documenta el desarrollo de una aplicación de gestión de países implementada en el lenguaje de programación **Python**, utilizando como soporte de almacenamiento un archivo **CSV (Comma Separated Values)**. El sistema permite registrar, visualizar, ordenar, filtrar y analizar información de países —incluyendo nombre, población, superficie y continente— a través de una interfaz de consola, con opción a extensión gráfica mediante la biblioteca **Tkinter**.

A lo largo del informe se abordan los **conceptos teóricos fundamentales** que sustentan el funcionamiento del programa: estructuras de datos, manejo de archivos, validación de entradas, persistencia de información y diseño modular. Posteriormente, se describe la **metodología empleada en la construcción del sistema**, el análisis funcional de cada módulo y las **estadísticas generadas** a partir de los datos cargados.

Finalmente, se presentan los **resultados obtenidos y las conclusiones generales**, destacando la correcta aplicación de los principios de estructuración de código, control de errores, reutilización de funciones y persistencia de datos.

Tabla de Contenido

Contenido

Resumen	1
1. Introducción.....	3
2. Marco Teórico.....	3
2.1. Lenguaje Python y paradigma estructurado.....	3
2.2. Manejo de archivos CSV	4
2.3. Estructuras de datos: listas y diccionarios	4
2.4. Validación de datos y control de errores.....	4
2.5. Tkinter como interfaz gráfica opcional	5
2.6. Conceptos Utilizados	5
Listas	5
Diccionarios.....	5
Funciones	6
Condicionales	6
Ordenamientos	6
Estadísticas básicas	6
Archivos CSV.....	7
3. Caso Práctico: Sistema de Gestión de Países.....	7
4. Metodología Utilizada	8
5. Resultados Obtenidos.....	9
6. Conclusiones.....	10
7. Bibliografía y Referencias.....	11

1. Introducción

En el ámbito del desarrollo de software, uno de los objetivos esenciales es la creación de programas que permitan **almacenar, manipular y analizar información** de manera eficiente y persistente. Este trabajo presenta el desarrollo de un **sistema de gestión de países**, implementado en Python, que permite registrar datos geográficos básicos, almacenarlos de forma permanente en un archivo CSV y realizar operaciones de filtrado, ordenamiento y análisis estadístico.

El sistema fue diseñado bajo una **arquitectura modular**, donde cada función cumple una responsabilidad específica, facilitando su mantenimiento y legibilidad. Asimismo, se priorizó la **validez y coherencia de los datos** mediante la implementación de controles de entrada que previenen errores comunes en el ingreso de información.

El propósito principal de este informe es **documentar la estructura, funcionamiento y fundamentos teóricos** del programa, demostrando cómo se integran distintos conceptos de programación estructurada, manipulación de archivos y análisis de datos, en una aplicación funcional, educativa y extensible.

2. Marco Teórico

2.1. Lenguaje Python y paradigma estructurado

Python es un lenguaje de programación interpretado, multiparadigma y de tipado dinámico, ampliamente utilizado por su simplicidad sintáctica y su enfoque en la legibilidad del código. En este proyecto se adoptó un **paradigma estructurado**, basado en la definición de funciones que encapsulan tareas específicas. Cada función del programa —como la carga, filtrado o ordenamiento de países— se comporta como un **módulo independiente**, favoreciendo la reutilización y el mantenimiento.

2.2. Manejo de archivos CSV

El formato CSV (Comma Separated Values) permite almacenar información tabular en texto plano, donde cada fila representa un registro y las columnas están separadas por comas.

El módulo csv de Python se utiliza para **leer y escribir datos de manera estructurada**, simplificando la persistencia de información sin necesidad de una base de datos compleja.

En este caso, el archivo paises.csv actúa como repositorio central de los datos del sistema.

El programa incluye una verificación inicial: si el archivo no existe, **se crea automáticamente con sus encabezados** (“nombre”, “poblacion”, “superficie”, “continente”). Este enfoque garantiza la **autonomía del script** y su capacidad de operar en cualquier entorno.

2.3. Estructuras de datos: listas y diccionarios

Los datos de los países son representados internamente mediante diccionarios, lo que permite acceder a sus atributos mediante claves (nombre, poblacion, superficie, continente).

La colección de todos los países se almacena en una lista de diccionarios, estructura que ofrece versatilidad para operaciones de filtrado, ordenamiento y análisis estadístico.

El uso combinado de listas y diccionarios ejemplifica el paradigma de datos estructurados en memoria, donde las operaciones son iterativas y fácilmente legibles.

2.4. Validación de datos y control de errores

Una parte crucial del programa es la verificación de la validez de las entradas del usuario.

El sistema implementa controles para evitar errores como:

- **Campos vacíos o numéricos en el nombre o continente.**
- **Valores negativos o no enteros en población y superficie.**
- **Duplicación de países en el archivo CSV.**

- **Ingreso de continentes no válidos.**

Estas validaciones previenen inconsistencias en el almacenamiento y garantizan que la información del archivo sea coherente. Además, se utilizan mensajes claros para guiar al usuario en caso de error, reforzando la usabilidad del sistema.

2.5. Tkinter como interfaz gráfica opcional

El código incluye la posibilidad de usar una **interfaz gráfica** mediante la biblioteca tkinter, estándar en Python. Aunque la implementación actual solo crea una ventana base (“Gestor de Países”), su inclusión demuestra la escalabilidad del proyecto y su potencial para evolucionar hacia una **aplicación visual completa**, con botones, formularios y tablas de datos.

2.6. Conceptos Utilizados

En el desarrollo del sistema “Gestor de Países en Python” se aplicaron diversos conceptos fundamentales de la programación estructurada y del análisis de datos. A continuación, se detallan los principales elementos teóricos implementados:

Listas

Las listas son estructuras de datos ordenadas y mutables que permiten almacenar múltiples elementos en una sola variable. En este proyecto, se utilizan para contener los registros de países cargados desde el archivo CSV. Cada elemento de la lista representa un país, lo que permite recorrerlos mediante bucles, aplicar filtros, realizar ordenamientos y calcular estadísticas.

Ejemplo:

```
lista_paises = [ {'nombre': 'Argentina', 'poblacion': 45000000}, {...}]
```

Diccionarios

Los diccionarios son estructuras de datos que almacenan pares clave-valor, permitiendo un acceso directo a los datos mediante una clave única. En el sistema, cada país se representa como un diccionario con sus atributos principales: nombre, población, superficie y

continente. Esto facilita la manipulación y búsqueda de información de manera eficiente.

Ejemplo:

```
pais = {  
    'nombre': 'Argentina',  
    'poblacion': 45000000,  
    'superficie': 2780000,  
    'continente': 'América'  
}
```

Funciones

Las funciones permiten agrupar bloques de código que realizan tareas específicas, favoreciendo la modularidad y la reutilización. El programa está estructurado en múltiples funciones independientes, como `agregar_pais()`, `mostrar_paises()`, `filtrar_paises()`, `ordenar_paises()` y `estadisticas_paises()`.

Esta organización mejora la legibilidad del código, evita repeticiones y facilita su mantenimiento o expansión futura.

Condicionales

Los condicionales (`if`, `elif`, `else`) se utilizan para tomar decisiones dentro del flujo del programa según ciertas condiciones. En este caso, se aplican para validar los datos ingresados, evitar duplicados, controlar opciones del menú y determinar criterios de filtrado u ordenamiento. Por ejemplo, se verifica que la población y superficie sean números positivos, o que el nombre del país no exista previamente en el archivo.

Ordenamientos

El programa implementa mecanismos de ordenamiento utilizando funciones integradas de Python como `sorted()` o métodos como `.sort()`. Los países pueden ordenarse por nombre, población o superficie, tanto en orden ascendente como descendente. El uso de claves de ordenamiento mediante funciones lambda (`key=lambda pais: pais['poblacion']`) permite una clasificación flexible y eficiente de los registros.

Estadísticas básicas

Se calculan estadísticas simples sobre los datos almacenados, como el promedio de población o superficie, el país con mayor y menor población, y la cantidad de países por continente. Estas operaciones se realizan mediante iteraciones sobre las listas y uso de

funciones como `max()`, `min()`, `sum()` y `len()`.

De esta forma, el sistema ofrece un análisis cuantitativo básico de la información gestionada.

Archivos CSV

El archivo CSV constituye el medio de persistencia del sistema, garantizando que los datos permanezcan disponibles entre ejecuciones. Gracias al módulo `csv` de Python, se pueden leer y escribir registros de forma estructurada, simulando una base de datos ligera. Cada operación del programa —agregar, mostrar, filtrar o analizar— interactúa directa o indirectamente con este archivo, asegurando la integridad y continuidad de la información.

3. Caso Práctico: Sistema de Gestión de Países

El programa inicia solicitando al usuario si desea usar la **interfaz gráfica o el modo consola**.

En caso de elegir el modo consola, se despliega un menú principal con seis opciones:

1. Añadir país
2. Mostrar países
3. Filtrar países
4. Ordenar países
5. Estadísticas
6. Salir

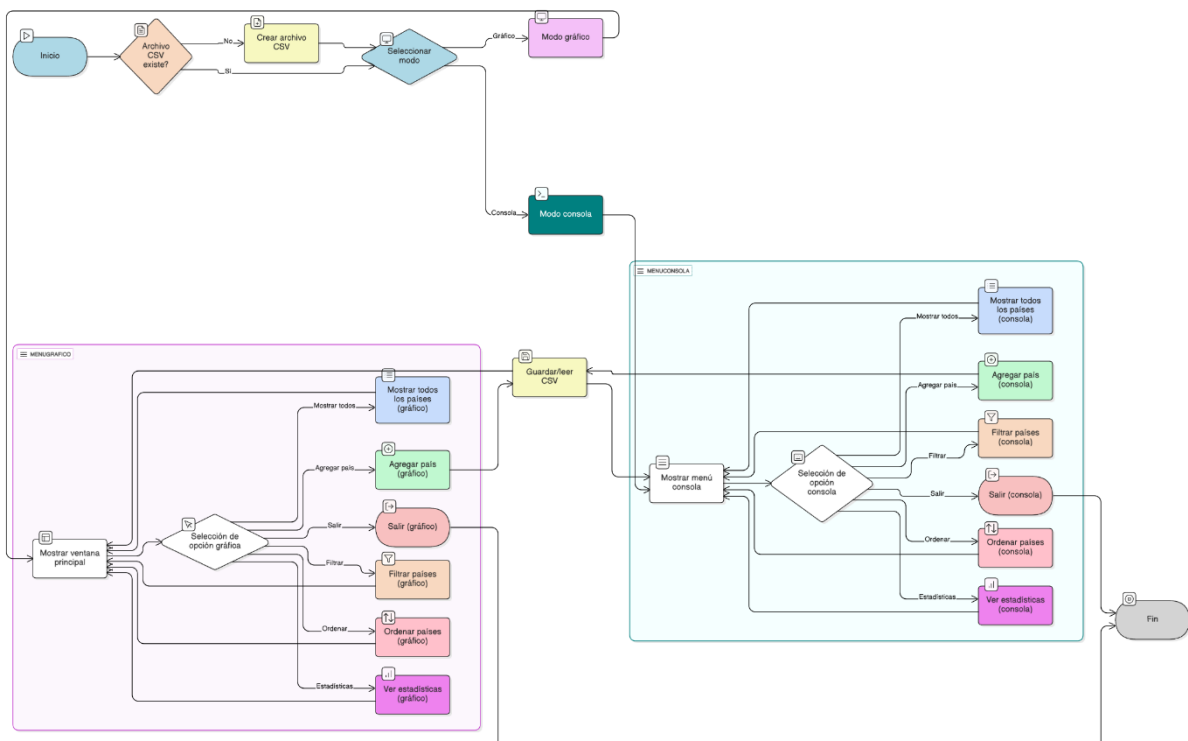
Cada opción ejecuta una función modular que realiza una operación específica:

- **Añadir País:** registra un nuevo país, validando datos y evitando duplicados.
- **Mostrar Países:** lista todos los países registrados en el CSV.

- **Filtrar Países:** permite búsquedas por continente, rango de población, superficie o coincidencia parcial de nombre.
- **Ordenar Países:** organiza la lista por una clave elegida, en orden ascendente o descendente.
- **Estadísticas:** calcula indicadores agregados, incluyendo país con mayor y menor población, promedios de población y superficie, y cantidad de países por continente.

Estas funciones demuestran una aplicación práctica de programación estructurada, condicionales, bucles e interacción con archivos externos, integrando teoría y práctica en un entorno funcional.

A continuación, agregamos una imagen del diagrama de flujo de lo que se busca realizar con el código



4. Metodología Utilizada

La metodología aplicada sigue un enfoque incremental y modular, característico de proyectos educativos de programación estructurada:

1. **Diseño del archivo CSV base:** creación automática con encabezados.
2. **Implementación de funciones independientes:** lectura, escritura, filtrado, ordenamiento y análisis.
3. **Integración de control de errores:** validaciones previas a cada ingreso de datos.
4. **Diseño de la interfaz de consola:** construcción de un menú interactivo y de fácil navegación.
5. **Incorporación opcional de Tkinter:** demostración de escalabilidad a entornos gráficos.

Cada bloque de código fue probado de forma independiente antes de integrarse en el programa principal. El objetivo fue lograr una aplicación estable, legible y reutilizable, que ejemplifique las buenas prácticas en el manejo de archivos y estructuras de datos.

5. Resultados Obtenidos

El desarrollo del sistema permitió alcanzar los siguientes resultados concretos:

- **Persistencia funcional:** todos los países se almacenan y recuperan correctamente desde el archivo CSV.
- **Eficiencia en el filtrado y ordenamiento:** las funciones permiten explorar los datos de manera dinámica, respondiendo a distintos criterios.
- **Cálculo de estadísticas automáticas:** la función `estadisticas_paises` resume información relevante de forma inmediata.
- **Robustez del sistema:** la inclusión de validaciones minimiza errores de ejecución y garantiza integridad de los datos.
- **Escalabilidad demostrada:** la base del sistema permite su futura migración a interfaz gráfica completa o incluso a una base de datos relacional.

El programa se destaca por su claridad estructural, su enfoque educativo y su valor como ejemplo integral de manejo de datos persistentes en Python.

6. Conclusiones

El análisis y desarrollo de este programa permitieron integrar múltiples conceptos teóricos y prácticos de la programación estructurada, el manejo de archivos y la persistencia de datos.

Se logró un sistema estable que cumple con los principios de modularidad, validación y reutilización, representando un modelo aplicable tanto en entornos educativos como en proyectos reales de pequeña escala.

Entre los principales aprendizajes se destacan:

- La importancia de la separación entre lógica de negocio y persistencia de datos.
- La utilidad de las estructuras de datos compuestas (listas y diccionarios) para representar información compleja.
- La necesidad del control de errores y validaciones en programas interactivos.
- El potencial de escalabilidad del lenguaje Python hacia entornos gráficos o conectados a bases de datos.

En conclusión, este trabajo demuestra que una correcta aplicación de los fundamentos de la programación y el manejo de archivos puede derivar en soluciones robustas, funcionales y fácilmente extensibles, cumpliendo con los objetivos académicos y prácticos propuestos.

7. Bibliografía y Referencias

- Python Software Foundation. (2025). *Python 3 Documentation*. Disponible en: <https://docs.python.org/3/>
- Python CSV Module. (2025). *Reading and Writing CSV Files in Python*. Disponible en: <https://docs.python.org/3/library/csv.html>
- Tkinter Official Guide. (2025). *Tkinter Reference Documentation*. Disponible en: <https://docs.python.org/3/library/tk.html>
- McKinney, W. (2018). *Python for Data Analysis (2nd ed.)*. O'Reilly Media.
- Sweigart, A. (2019). *Automate the Boring Stuff with Python*. No Starch Press.
- Lundh, F. (2005). *An Introduction to Tkinter*. PythonWare.
- Lutz, M. (2013). *Learning Python (5th ed.)*. O'Reilly Media.